

Edge cases (a bad release that slips past the gate)

— Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `release_gate_practice.py` (Edge cases section): a gate that can be skipped by editing one file isn't a gate. This section proves the bypass is real, then closes it — first by signing the pointer, then by removing the separate pointer altogether. **If not:** the Build Task's constraint ("the gate must not be skippable by accident") would be a hope, not something you'd tested.

Every approach below starts by proving the bypass is real, then fixes it a different way.

Basic Version

Approach 1 — prove the bypass, then a plain hash check

Story: first show that nothing stops a hand-edit, then add a check that catches a careless one. **If not:** you'd assume the gate was safe because you never tried to get around it.

```
# release_gate_practice.py – Edge cases section
import hashlib
import json

# --- step 1: prove the bypass is real ---
with open("live_version.txt", "w") as f:
    # v1 never passed the 0.9 threshold in release_gate – nothing stopped this
    f.write("v1")

with open("live_version.txt") as f:
    print("live (before any fix):", f.read())
```

Expected output:

```
live (before any fix): v1
```

Nothing checked whether `v1` actually passed the gate — it's just a text file, and text files don't know the difference.

```
# release_gate_practice.py – Edge cases section
# --- step 2: a plain hash check ---
import logging

logging.basicConfig(level=logging.INFO)

class TamperedPointerError(Exception):
    pass

def deploy_and_save_pointer(version_name):
    # imagine this only runs after deploy()'s score check passed
    check = hashlib.sha256(version_name.encode()).hexdigest()
    with open("live_version.json", "w") as f:
        json.dump({"version_name": version_name, "check": check}, f)

def get_live_version():
    with open("live_version.json") as f:
        pointer = json.load(f)
    expected = hashlib.sha256(pointer["version_name"].encode()).hexdigest()
    if pointer["check"] != expected:
        logging.critical(
            "Live pointer looks tampered with! live_version.json says "
            + pointer["version_name"]
        )
        raise TamperedPointerError(
            "live_version.json failed its check - refusing to trust it"
        )
    return pointer["version_name"]

deploy_and_save_pointer("v2")
print(get_live_version())
```

Expected output:

```
v2
```

Now hand-edit `live_version.json`, changing `"version_name"` to `"v1"` but leaving `"check"` as it was:

```
print(get_live_version())
```

Expected output:

```
CRITICAL:root:Live pointer looks tampered with! live_version.json says v1
Traceback (most recent call last):
...
__main__.TamperedPointerError: live_version.json failed its check - refusing
to trust it
```

This version works correctly for what the exercise asks — a careless hand-edit is now caught. It's not safe against someone who reads this same script, though:

`hashlib.sha256(version_name.encode())` uses no secret, so anyone who can see this code can recompute a matching "check" for any version name they like. That's what Intermediate fixes.

Revision from Doc01 (Basic): a detected bypass is logged with `logging.critical(...)` and then stops with a simple named error, `TamperedPointerError`. `CRITICAL` fits Doc01's definition exactly: if the pointer can't be trusted, the program no longer knows which version should serve users, so it can't keep going as it is. The raise matters as much as the log: returning `None` (or printing a warning) lets the caller carry on, maybe treating `None` as "nothing is live" and deploying anyway. A named error stops everything, and its name tells the reader what went wrong.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — `hmac`, signed with a secret

Story: a plain hash can be recomputed by anyone who reads the code. A signature that needs a secret from `.env` can't. **If not:** a teammate who saw the script could "fix" the check along with the pointer.

```
# release_gate_practice.py – Edge cases section
import hmac
import hashlib
import json
import logging
import os
from dotenv import load_dotenv

logger = logging.getLogger(__name__)
logger.setLevel(logging.INFO)
logger.addHandler(logging.StreamHandler())

class MissingSecretError(Exception):
    pass

class TamperedPointerError(Exception):
    pass

load_dotenv()
SECRET = os.getenv("DEPLOY_GATE_SECRET")
if SECRET is None or SECRET == "":
    raise MissingSecretError(
        "DEPLOY_GATE_SECRET is not set in .env – "
        "pointers can't be signed or checked without it"
    )

def sign(version_name: str) -> str:
    # how: the signature depends on the name AND the secret
    key = SECRET.encode()
    message = version_name.encode()
    return hmac.new(key, message, hashlib.sha256).hexdigest()

def save_pointer(version_name: str) -> None:
```

```

pointer = {"version_name": version_name, "signature": sign(version_name)}
with open("live_version.json", "w") as f:
    json.dump(pointer, f)

def get_live_version() -> str:
    with open("live_version.json") as f:
        pointer = json.load(f)
    try:
        version_name = pointer["version_name"]
        stored_signature = pointer["signature"]
    except KeyError as e:
        logger.critical(f"Live pointer is missing the field {e}")
        raise TamperedPointerError(
            f"live_version.json has no {e} field – refusing to trust it"
        )

    if not hmac.compare_digest(stored_signature, sign(version_name)):
        logger.critical(
            f"Live pointer signature does not match its version name: "
            f"{version_name}"
        )
        raise TamperedPointerError(
            "live_version.json failed its signature check – "
            "refusing to trust it"
        )
    return version_name

save_pointer("v2")
print(get_live_version())

```

Expected output:

```
v2
```

Hand-edit `live_version.json`'s `"version_name"` to `"v1"` without recomputing `"signature"` :

```
print(get_live_version())
```

Expected output:

```

Live pointer signature does not match its version name: v1
Traceback (most recent call last):
...

```

```
__main__.TamperedPointerError: live_version.json failed its signature check –
refusing to trust it
```

(the first line is the `CRITICAL` log — this handler has no formatter, so only the message shows). If someone deletes the `"signature"` field instead, the `KeyError` is caught and you get a `TamperedPointerError` naming the missing field, not a bare `KeyError: 'signature'`.

Unlike Basic, an attacker who has read this script still can't produce a matching `"signature"` for `"v1"` without also knowing `SECRET` — `hmac.new` ties the signature to both the version name *and* a value that isn't written down inside `live_version.json` itself.

Revision from Doc01 (Intermediate): two habits come back. First, `load_dotenv()` plus the "required value is missing → raise a named error" rule: the secret comes from `.env`, and if it isn't there, the script stops at startup with `MissingSecretError`. A default secret written in the code would defeat the whole idea — anyone reading the code would know it. Second, catching one *specific* exception, `KeyError`, and raising a named `TamperedPointerError` with a clear message: a pointer with a deleted field is a tampered pointer too, and it should be reported as one, through a named logger at `CRITICAL`.

Difference from Basic: Basic's plain `sha256` hash catches a careless hand-edit but is trivially fakeable by anyone who's read the code, since nothing about it is secret. Intermediate's `hmac.new(SECRET.encode(), ...)` depends on a value that lives only in `.env` (read with `load_dotenv()`, and required at startup) — never inside the code or the pointer file that gets edited — so recomputing a valid signature requires knowing that secret, not just the hashing algorithm. `hmac.compare_digest` also replaces a plain `==`, avoiding a timing side-channel that a naive string comparison has (a minor point for a local file, a real one for anything checked over a network).

Approach 2 — remove the separate pointer file; the version log is the only source of truth

Story: signing a pointer file protects it, but the pointer is still a second record that can drift from the truth. Removing it — and working out "what's live" from the version log itself — leaves only one place to trust. **If not:** you'd keep two records of the same fact, and one day they'd disagree.

```
# release_gate_practice.py – Edge cases section
import json
import time
from datetime import datetime, timezone
from pathlib import Path
import logging

logger = logging.getLogger(__name__)
logger.setLevel(logging.INFO)
logger.addHandler(logging.StreamHandler())
```

```

class VersionLogCorruptError(Exception):
    pass

def load_versions(path: str = "versions.json") -> list[dict]:
    if not Path(path).exists():
        return []
    with open(path) as f:
        try:
            return json.load(f)
        except json.JSONDecodeError as e:
            # why: this log is now the ONLY record of what's live –
            # a damaged one is the worst failure this code can have
            logger.critical(
                "Version log %s is unreadable (line %d, column %d)",
                path, e.lineno, e.colno,
            )
            raise VersionLogCorruptError(
                f"{path} is not valid JSON – restore it from backup"
            ) from e

def save_versions(versions: list[dict], path: str = "versions.json") -> None:
    with open(path, "w") as f:
        json.dump(versions, f, indent=2)

def save_version(
    name: str, prompt_text: str, path: str = "versions.json"
) -> None:
    versions = load_versions(path)
    versions.append({
        "version_name": name,
        "prompt_text": prompt_text,
        "created_at": datetime.now(timezone.utc).isoformat(),
        "passed_gate": False,
    })
    save_versions(versions, path)

def mark_passed(version_name: str, path: str = "versions.json") -> None:
    versions = load_versions(path)
    for v in versions:
        if v["version_name"] == version_name:
            v["passed_gate"] = True
            logger.info("Marked %s as passed_gate", version_name)
    save_versions(versions, path)

def get_live_version(path: str = "versions.json") -> str | None:
    # how: "live" is worked out from the log every time – there is no
    # second file that could disagree with it or be edited on its own

```

```

versions = load_versions(path)
passing = []
for v in versions:
    if v.get("passed_gate"):
        passing.append(v)

if not passing:
    return None

latest = passing[0]
for v in passing:
    if v["created_at"] > latest["created_at"]:
        latest = v
return latest["version_name"]

save_version("v1", "Answer the customer's question.")
time.sleep(0.01)
save_version(
    "v2", "Answer the customer's question politely and cite the source."
)

print("live before anything passes:", get_live_version())

# imagine this only runs inside deploy(), after the score check passed
mark_passed("v2")
print("live after v2 passes:", get_live_version())

```

Expected output:

```

live before anything passes: None
Marked v2 as passed_gate
live after v2 passes: v2

```

(the middle line is a log line — this handler has no formatter.)

There's no `live_version.txt` or `live_version.json` left at all — `get_live_version()` derives the answer straight from the version log's own `passed_gate` field, the exact same record `deploy()` already writes.

A determined hand-edit of `versions.json` itself (setting another version's `passed_gate` to `true`) is still possible — the win is that there's no separate side door any more. The Build Task goes one step further: every gate decision becomes its own timestamped event in an append-only history, so "what was live at time T" can be answered too.

Difference between Approach 1 and Approach 2: Approach 1 makes a separate pointer file's claim checkable with a secret. Approach 2 asks whether that separate file needs to exist at all — and

removes it, so "what's live" is always derived from the one log that `deploy()` already writes.

Which one should you actually write? Approach 2's idea — one source of truth, with "live" worked out from it — is the shape the Build Task uses. Add Approach 1's `hmac` signature on top once more than one person can write to that file.